

Simulations

2026-03-31

Goal: test data simulations

This script runs simulations on applicant data to determine most likely job title and city

Define simulation parameters

```
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.2.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

Candidates <- read.csv("School_Front (3-30).csv")

set.seed(123)
runs <- 1000

cities <- c("Canandaigua", "Rochester")

jobs <- c("Certified Positions (Misc)", "Assistant Principal",
          "HR Assistant", "Teaching Assistant")

n_candidates <- nrow(Candidates)
```

Randomly assign a city and job to each candidate

```
sim_results <- list()

for (i in 1:runs) {

  assigned_city <- sample(cities, n_candidates, replace = TRUE)
```

```

assigned_job <- sample(jobs, n_candidates, replace = TRUE)

# Count how many candidates fall into each city/job combination
for (city in cities) {
  for (job in jobs) {
    count <- sum(assigned_city == city & assigned_job == job)

    sim_results[[length(sim_results) + 1]] <- data.frame(
      Simulation = i,
      City       = city,
      Job        = job,
      Count      = count
    )
  }
}

# Combine all results into a single data frame
sim_results_df <- dplyr::bind_rows(sim_results)

```

No loops option

```

sim_list <- replicate(
  runs,
  {
    assigned_city <- sample(cities, n_candidates, replace = TRUE)
    assigned_job  <- sample(jobs, n_candidates, replace = TRUE)
    as.data.frame(table(assigned_city, assigned_job))
  },
  simplify = FALSE
)

sim_results_df <- bind_rows(sim_list, .id = "Simulation")

# Optional: rename columns for clarity
sim_results_df <- sim_results_df %>%
  rename(
    City = assigned_city,
    Job  = assigned_job,
    Count = Freq
  )

sim_results_df %>%
  group_by(City, Job) %>%
  summarise(Average_Count = mean(Count), .groups = "drop")

```

```

## # A tibble: 8 x 3
##   City          Job          Average_Count
##   <fct>        <fct>          <dbl>
## 1 Canandaigua Assistant Principal      737.
## 2 Canandaigua Certified Positions (Misc) 736.

```

```
## 3 Canandaigua HR Assistant 735.
## 4 Canandaigua Teaching Assistant 734.
## 5 Rochester Assistant Principal 736.
## 6 Rochester Certified Positions (Misc) 737.
## 7 Rochester HR Assistant 734.
## 8 Rochester Teaching Assistant 735.
```

```
sim_results_df %>%
  dplyr::group_by(City, Job) %>%
  dplyr::summarise(Average_Count = mean(Count))
```

'summarise()' has grouped output by 'City'. You can override using the
'.groups' argument.

```
## # A tibble: 8 x 3
## # Groups:   City [2]
##   City      Job      Average_Count
##   <fct>    <fct>          <dbl>
## 1 Canandaigua Assistant Principal 737.
## 2 Canandaigua Certified Positions (Misc) 736.
## 3 Canandaigua HR Assistant 735.
## 4 Canandaigua Teaching Assistant 734.
## 5 Rochester Assistant Principal 736.
## 6 Rochester Certified Positions (Misc) 737.
## 7 Rochester HR Assistant 734.
## 8 Rochester Teaching Assistant 735.
```